import libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, confusion_matrix
from sklearn.datasets import load_breast_cancer # Import
load_breast_cancer

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.datasets import mnist, cifar10
```

load datsets

```
data = load_breast_cancer()

X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target

print("Dataset Shape:", X.shape)
X.head()
```

```
Dataset Shape: (569, 30)
```

{"type":"dataframe","variable_name":"X"}

```
# ============================
# BREAST CANCER PIPELINE
# ============================

from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from tensorflow.keras import models, layers

scaler = StandardScaler()
X_bc = scaler.fit_transform(X)

X_train_bc, X_test_bc, y_train_bc, y_test_bc = train_test_split(X_bc,
y, test_size=0.2)

model_bc = models.Sequential([
    layers.Dense(64, activation='relu'),
    layers.Dense(32, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])
```

```
model_bc.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

history_bc = model_bc.fit(X_train_bc, y_train_bc, epochs=10,
validation_data=(X_test_bc, y_test_bc))

bc_acc = model_bc.evaluate(X_test_bc, y_test_bc)[1]
```

```
Epoch 1/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8391 - loss:
0.3457 - val_accuracy: 0.8512 - val_loss: 0.3258
Epoch 2/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8530 - loss:
0.3145 - val_accuracy: 0.8561 - val_loss: 0.3208
Epoch 3/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8553 - loss:
0.3082 - val_accuracy: 0.8547 - val_loss: 0.3191
Epoch 4/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8591 - loss:
0.3031 - val_accuracy: 0.8548 - val_loss: 0.3166
Epoch 5/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 8s 2ms/step - accuracy: 0.8605 - loss:
0.3000 - val_accuracy: 0.8553 - val_loss: 0.3175
Epoch 6/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.8611 - loss:
0.2970 - val_accuracy: 0.8579 - val_loss: 0.3182
Epoch 7/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 2ms/step - accuracy: 0.8626 - loss:
0.2950 - val_accuracy: 0.8552 - val_loss: 0.3190
Epoch 8/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8632 - loss:
0.2925 - val_accuracy: 0.8546 - val_loss: 0.3173
Epoch 9/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8641 - loss:
0.2907 - val_accuracy: 0.8555 - val_loss: 0.3174
Epoch 10/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 7s 6ms/step - accuracy: 0.8664 - loss:
0.2880 - val_accuracy: 0.8578 - val_loss: 0.3209
306/306 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8578 - loss:
0.3209
```

```
df = pd.read_csv('/content/adult.csv', na_values=" ?")

# Drop missing values
df.dropna(inplace=True)

# Convert categorical to numeric
df = pd.get_dummies(df, drop_first=True)
```

```
X = df.drop("income_>50K", axis=1)
y = df["income_>50K"]

print("Dataset Shape:", X.shape)
X.head()
```

```
Dataset Shape: (48842, 100)
```

{"type":"dataframe","variable_name":"X"}

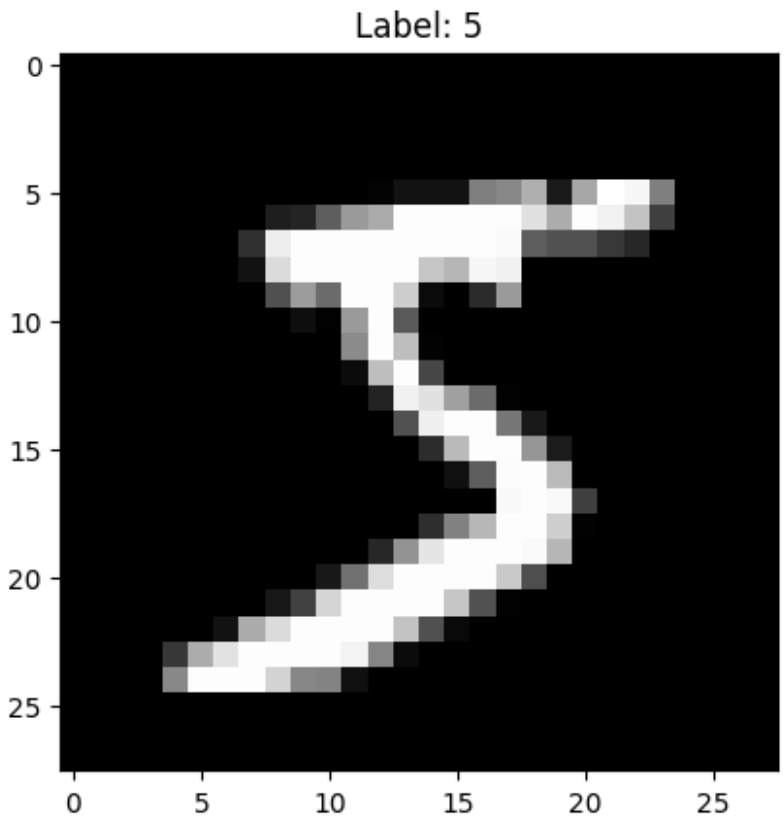
```
# ============================
# ADULT PIPELINE
# ============================

scaler = StandardScaler()
X_ad = scaler.fit_transform(X)

X_train_ad, X_test_ad, y_train_ad, y_test_ad = train_test_split(X_ad,
y, test_size=0.2)

model_ad = models.Sequential([
    layers.Dense(128, activation='relu'),
    layers.Dense(64, activation='relu'),
    layers.Dense(1, activation='sigmoid')
])

model_ad.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])

history_ad = model_ad.fit(X_train_ad, y_train_ad, epochs=10,
validation_data=(X_test_ad, y_test_ad))

ad_acc = model_ad.evaluate(X_test_ad, y_test_ad)[1]
```

```
Epoch 1/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 6s 4ms/step - accuracy: 0.8418 - loss:
0.3430 - val_accuracy: 0.8579 - val_loss: 0.3152
Epoch 2/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8537 - loss:
0.3132 - val_accuracy: 0.8565 - val_loss: 0.3082
Epoch 3/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8577 - loss:
0.3064 - val_accuracy: 0.8584 - val_loss: 0.3093
Epoch 4/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 6s 3ms/step - accuracy: 0.8594 - loss:
0.3019 - val_accuracy: 0.8572 - val_loss: 0.3123
Epoch 5/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8614 - loss:
0.2989 - val_accuracy: 0.8588 - val_loss: 0.3095
Epoch 6/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8637 - loss:
```

```
0.2940 - val_accuracy: 0.8583 - val_loss: 0.3078
Epoch 7/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 5s 3ms/step - accuracy: 0.8639 - loss:
0.2918 - val_accuracy: 0.8589 - val_loss: 0.3143
Epoch 8/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 4ms/step - accuracy: 0.8667 - loss:
0.2872 - val_accuracy: 0.8585 - val_loss: 0.3104
Epoch 9/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8687 - loss:
0.2841 - val_accuracy: 0.8579 - val_loss: 0.3122
Epoch 10/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 3ms/step - accuracy: 0.8696 - loss:
0.2799 - val_accuracy: 0.8562 - val_loss: 0.3130
306/306 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8562 - loss:
0.3130
```

```python
from tensorflow.keras.datasets import mnist

(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Combine (optional, for uniform style like yours)
X = np.concatenate((X_train, X_test), axis=0)
y = np.concatenate((y_train, y_test), axis=0)

print("Dataset Shape:", X.shape)
print("Sample Label:", y[0])

# Show first image
plt.imshow(X[0], cmap='gray')
plt.title(f"Label: {y[0]}")
plt.show()
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Dataset Shape: (70000, 28, 28)
Sample Label: 5
```



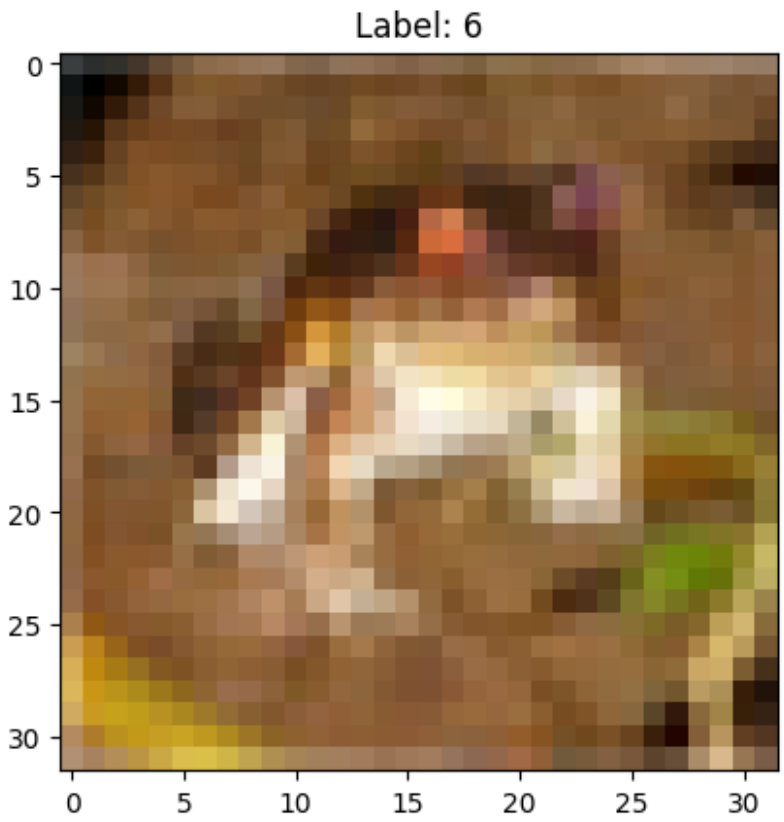

```
# ============================
# MNIST PIPELINE
# ============================

from tensorflow.keras.datasets import mnist

(X_train_mnist, y_train_mnist), (X_test_mnist, y_test_mnist) =
mnist.load_data()

X_train_m = X_train_mnist / 255.0
X_test_m = X_test_mnist / 255.0

X_train_m = X_train_m.reshape(-1,28,28,1)
X_test_m = X_test_m.reshape(-1,28,28,1)

model_mnist = models.Sequential([
    layers.Conv2D(32,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),
    layers.Conv2D(64,(3,3),activation='relu'),
    layers.MaxPooling2D(2,2),
    layers.Flatten(),
    layers.Dense(64,activation='relu'),
    layers.Dense(10,activation='softmax')
])
```

```
model_mnist.compile(optimizer='adam',
                    loss='sparse_categorical_crossentropy',
                    metrics=['accuracy'])

history_mnist = model_mnist.fit(X_train_m, y_train_mnist, epochs=5,
validation_data=(X_test_m, y_test_mnist))

mnist_acc = model_mnist.evaluate(X_test_m, y_test_mnist)[1]
```

```
Epoch 1/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 64s 33ms/step - accuracy: 0.9592 -
loss: 0.1362 - val_accuracy: 0.9832 - val_loss: 0.0475
Epoch 2/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 79s 31ms/step - accuracy: 0.9859 -
loss: 0.0450 - val_accuracy: 0.9880 - val_loss: 0.0356
Epoch 3/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 59s 31ms/step - accuracy: 0.9900 -
loss: 0.0315 - val_accuracy: 0.9883 - val_loss: 0.0332
Epoch 4/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 60s 32ms/step - accuracy: 0.9927 -
loss: 0.0232 - val_accuracy: 0.9899 - val_loss: 0.0303
Epoch 5/5
1875/1875 ━━━━━━━━━━━━━━━━━━━━ 61s 32ms/step - accuracy: 0.9944 -
loss: 0.0173 - val_accuracy: 0.9878 - val_loss: 0.0363
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9878 - loss:
0.0363
```

```
from tensorflow.keras.datasets import cifar10

(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Combine datasets
X = np.concatenate((X_train, X_test), axis=0)
y = np.concatenate((y_train, y_test), axis=0)

print("Dataset Shape:", X.shape)
print("Sample Label:", y[0])

# Show sample image
plt.imshow(X[0])
plt.title(f"Label: {y[0][0]}")
plt.show()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-
python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 2s 0us/step
Dataset Shape: (60000, 32, 32, 3)
Sample Label: [6]
```



EDA (Exploratory Data Analysis)

For Breast Cancer & Adult (Tabular Data)

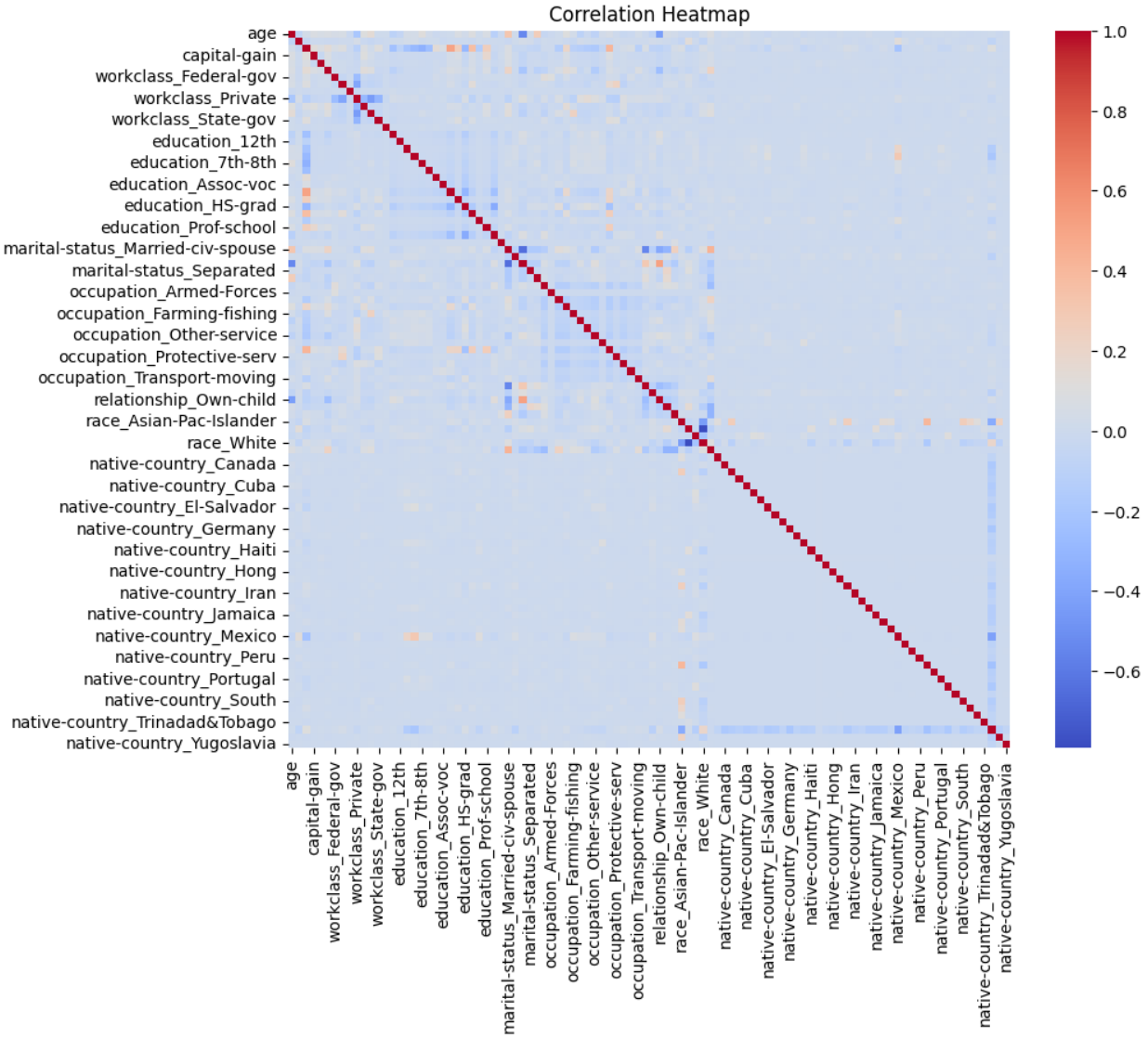
```
# Re-assign X and y to the adult dataset for tabular EDA
df_adult = pd.read_csv('/content/adult.csv', na_values=" ?")
df_adult.dropna(inplace=True)
df_adult = pd.get_dummies(df_adult, drop_first=True)
X = df_adult.drop("income_>50K", axis=1)
y = df_adult["income_>50K"]

print(X.info())
print(X.describe())

# Check missing values
print(X.isnull().sum())

# Correlation Heatmap
plt.figure(figsize=(10,8))
sns.heatmap(X.corr(), cmap='coolwarm')
plt.title("Correlation Heatmap")
plt.show()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48842 entries, 0 to 48841
```

```
Data columns (total 100 columns):
 #   Column                                    Non-Null Count  Dtype
---  ------                                    --------------  -----
 0   age                                       48842 non-null  int64
 1   fnlwgt                                    48842 non-null  int64
 2   educational-num                           48842 non-null  int64
 3   capital-gain                              48842 non-null  int64
 4   capital-loss                              48842 non-null  int64
 5   hours-per-week                            48842 non-null  int64
 6   workclass_Federal-gov                     48842 non-null  bool
 7   workclass_Local-gov                       48842 non-null  bool
 8   workclass_Never-worked                    48842 non-null  bool
 9   workclass_Private                         48842 non-null  bool
 10  workclass_Self-emp-inc                    48842 non-null  bool
 11  workclass_Self-emp-not-inc                48842 non-null  bool
 12  workclass_State-gov                       48842 non-null  bool
 13  workclass_Without-pay                     48842 non-null  bool
 14  education_11th                            48842 non-null  bool
 15  education_12th                            48842 non-null  bool
 16  education_1st-4th                         48842 non-null  bool
 17  education_5th-6th                         48842 non-null  bool
 18  education_7th-8th                         48842 non-null  bool
 19  education_9th                             48842 non-null  bool
 20  education_Assoc-acdm                      48842 non-null  bool
 21  education_Assoc-voc                       48842 non-null  bool
 22  education_Bachelors                       48842 non-null  bool
 23  education_Doctorate                       48842 non-null  bool
 24  education_HS-grad                         48842 non-null  bool
 25  education_Masters                         48842 non-null  bool
 26  education_Preschool                       48842 non-null  bool
 27  education_Prof-school                     48842 non-null  bool
 28  education_Some-college                    48842 non-null  bool
 29  marital-status_Married-AF-spouse          48842 non-null  bool
 30  marital-status_Married-civ-spouse         48842 non-null  bool
 31  marital-status_Married-spouse-absent      48842 non-null  bool
 32  marital-status_Never-married              48842 non-null  bool
 33  marital-status_Separated                  48842 non-null  bool
 34  marital-status_Widowed                    48842 non-null  bool
 35  occupation_Adm-clerical                   48842 non-null  bool
 36  occupation_Armed-Forces                   48842 non-null  bool
 37  occupation_Craft-repair                   48842 non-null  bool
 38  occupation_Exec-managerial                48842 non-null  bool
 39  occupation_Farming-fishing                48842 non-null  bool
 40  occupation_Handlers-cleaners              48842 non-null  bool
 41  occupation_Machine-op-inspct              48842 non-null  bool
 42  occupation_Other-service                  48842 non-null  bool
 43  occupation_Priv-house-serv                48842 non-null  bool
 44  occupation_Prof-specialty                 48842 non-null  bool
 45  occupation_Protective-serv                48842 non-null  bool
```

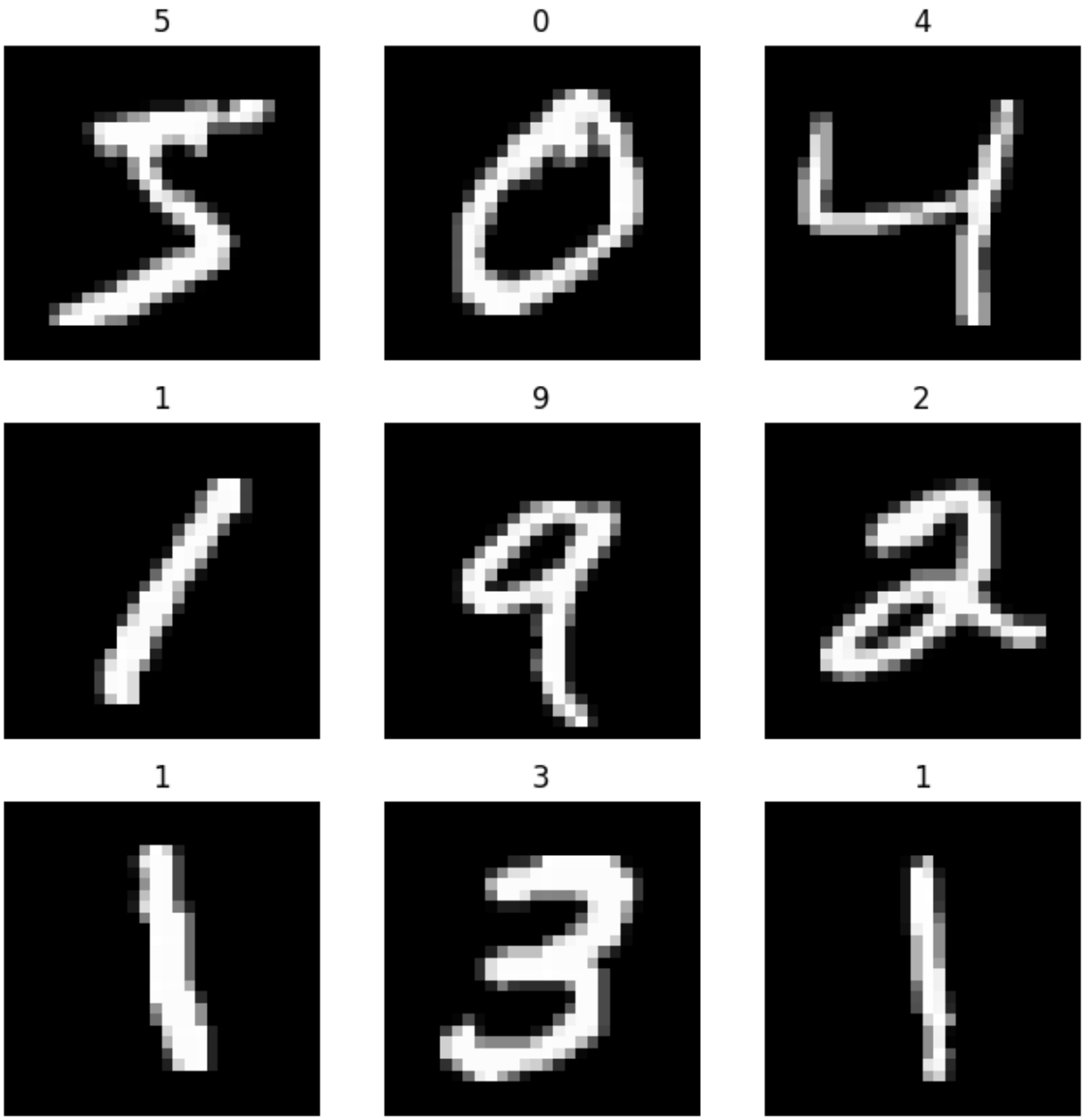
```
46  occupation_Sales                          48842 non-null  bool
47  occupation_Tech-support                   48842 non-null  bool
48  occupation_Transport-moving               48842 non-null  bool
49  relationship_Not-in-family                48842 non-null  bool
50  relationship_Other-relative               48842 non-null  bool
51  relationship_Own-child                    48842 non-null  bool
52  relationship_Unmarried                    48842 non-null  bool
53  relationship_Wife                         48842 non-null  bool
54  race_Asian-Pac-Islander                   48842 non-null  bool
55  race_Black                                48842 non-null  bool
56  race_Other                                48842 non-null  bool
57  race_White                                48842 non-null  bool
58  gender_Male                               48842 non-null  bool
59  native-country_Cambodia                   48842 non-null  bool
60  native-country_Canada                     48842 non-null  bool
61  native-country_China                      48842 non-null  bool
62  native-country_Columbia                   48842 non-null  bool
63  native-country_Cuba                       48842 non-null  bool
64  native-country_Dominican-Republic         48842 non-null  bool
65  native-country_Ecuador                    48842 non-null  bool
66  native-country_El-Salvador                48842 non-null  bool
67  native-country_England                    48842 non-null  bool
68  native-country_France                     48842 non-null  bool
69  native-country_Germany                    48842 non-null  bool
70  native-country_Greece                     48842 non-null  bool
71  native-country_Guatemala                  48842 non-null  bool
72  native-country_Haiti                      48842 non-null  bool
73  native-country_Holand-Netherlands         48842 non-null  bool
74  native-country_Honduras                   48842 non-null  bool
75  native-country_Hong                       48842 non-null  bool
76  native-country_Hungary                    48842 non-null  bool
77  native-country_India                      48842 non-null  bool
78  native-country_Iran                       48842 non-null  bool
79  native-country_Ireland                    48842 non-null  bool
80  native-country_Italy                      48842 non-null  bool
81  native-country_Jamaica                    48842 non-null  bool
82  native-country_Japan                      48842 non-null  bool
83  native-country_Laos                       48842 non-null  bool
84  native-country_Mexico                     48842 non-null  bool
85  native-country_Nicaragua                  48842 non-null  bool
86  native-country_Outlying-US(Guam-USVI-etc) 48842 non-null  bool
87  native-country_Peru                       48842 non-null  bool
88  native-country_Philippines                48842 non-null  bool
89  native-country_Poland                     48842 non-null  bool
90  native-country_Portugal                   48842 non-null  bool
91  native-country_Puerto-Rico                48842 non-null  bool
92  native-country_Scotland                   48842 non-null  bool
93  native-country_South                      48842 non-null  bool
94  native-country_Taiwan                     48842 non-null  bool
```

```
 95  native-country_Thailand                    48842 non-null  bool
 96  native-country_Trinadad&Tobago             48842 non-null  bool
 97  native-country_United-States               48842 non-null  bool
 98  native-country_Vietnam                     48842 non-null  bool
 99  native-country_Yugoslavia                  48842 non-null  bool
dtypes: bool(94), int64(6)
memory usage: 6.6 MB
None
                age        fnlwgt  educational-num  capital-gain  \
count  48842.000000  4.884200e+04     48842.000000  48842.000000
mean      38.643585  1.896641e+05        10.078089   1079.067626
std       13.710510  1.056040e+05         2.570973   7452.019058
min       17.000000  1.228500e+04         1.000000      0.000000
25%       28.000000  1.175505e+05         9.000000      0.000000
50%       37.000000  1.781445e+05        10.000000      0.000000
75%       48.000000  2.376420e+05        12.000000      0.000000
max       90.000000  1.490400e+06        16.000000  99999.000000

       capital-loss  hours-per-week
count  48842.000000    48842.000000
mean      87.502314       40.422382
std      403.004552       12.391444
min        0.000000        1.000000
25%        0.000000       40.000000
50%        0.000000       40.000000
75%        0.000000       45.000000
max     4356.000000       99.000000
age                              0
fnlwgt                           0
educational-num                  0
capital-gain                     0
capital-loss                     0
                                ..
native-country_Thailand          0
native-country_Trinadad&Tobago   0
native-country_United-States     0
native-country_Vietnam           0
native-country_Yugoslavia        0
Length: 100, dtype: int64
```



For MNIST

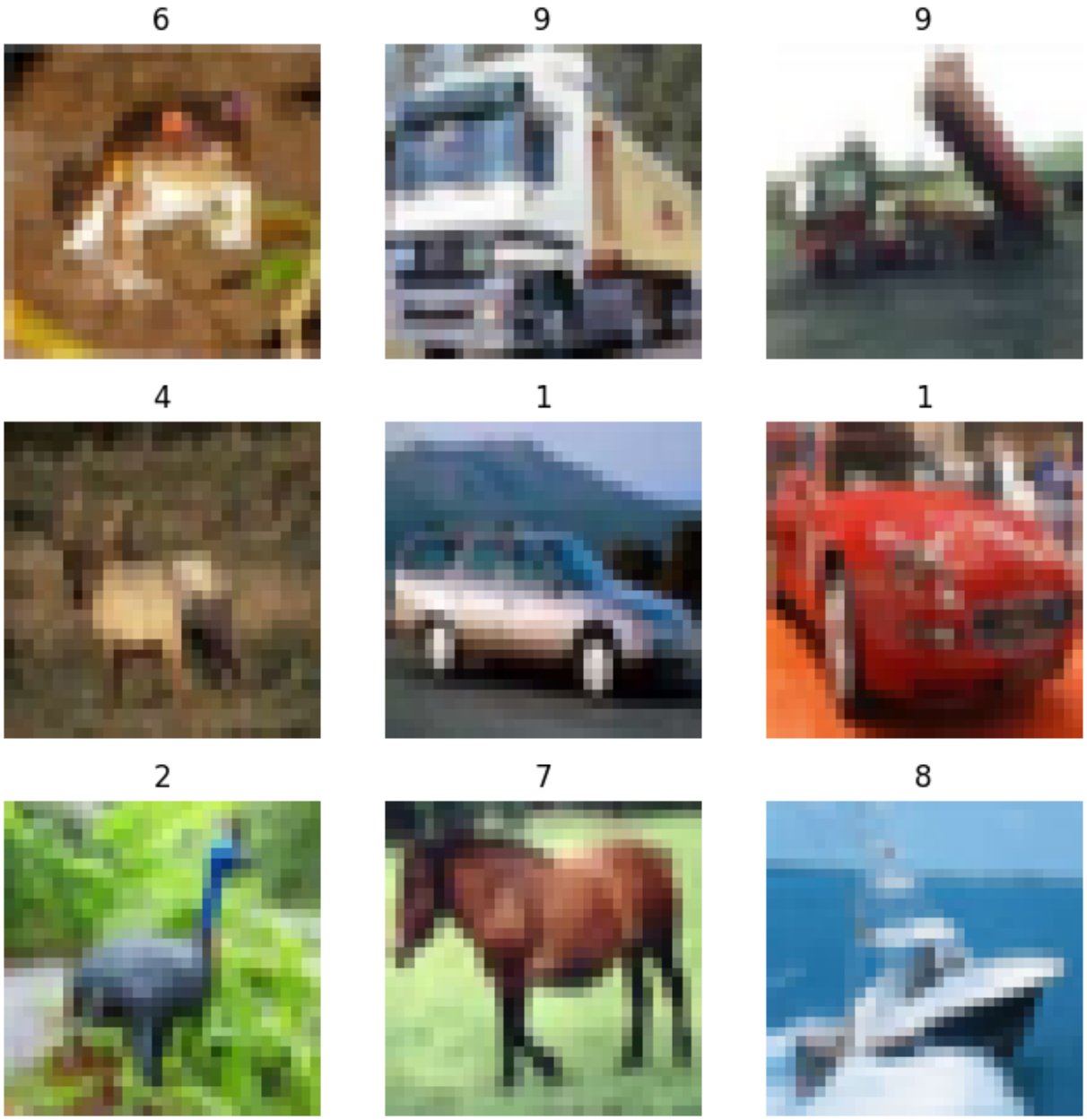
```
# Re-load MNIST dataset for image display
(X_train_mnist, y_train_mnist), (X_test_mnist, y_test_mnist) =
mnist.load_data()
X_mnist = np.concatenate((X_train_mnist, X_test_mnist), axis=0)
y_mnist = np.concatenate((y_train_mnist, y_test_mnist), axis=0)

# Show multiple images
plt.figure(figsize=(8,8))
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X_mnist[i], cmap='gray')
    plt.title(y_mnist[i])
    plt.axis('off')
plt.show()
```



For CIFAR-10

```
from tensorflow.keras.datasets import cifar10

(X_train_cifar, y_train_cifar), (X_test_cifar, y_test_cifar) =
cifar10.load_data()

# Combine datasets for consistent plotting
X = np.concatenate((X_train_cifar, X_test_cifar), axis=0)
y = np.concatenate((y_train_cifar, y_test_cifar), axis=0)

plt.figure(figsize=(8,8))
```

```
for i in range(9):
    plt.subplot(3,3,i+1)
    plt.imshow(X[i])
    plt.title(y[i][0])
    plt.axis('off')
plt.show()
```

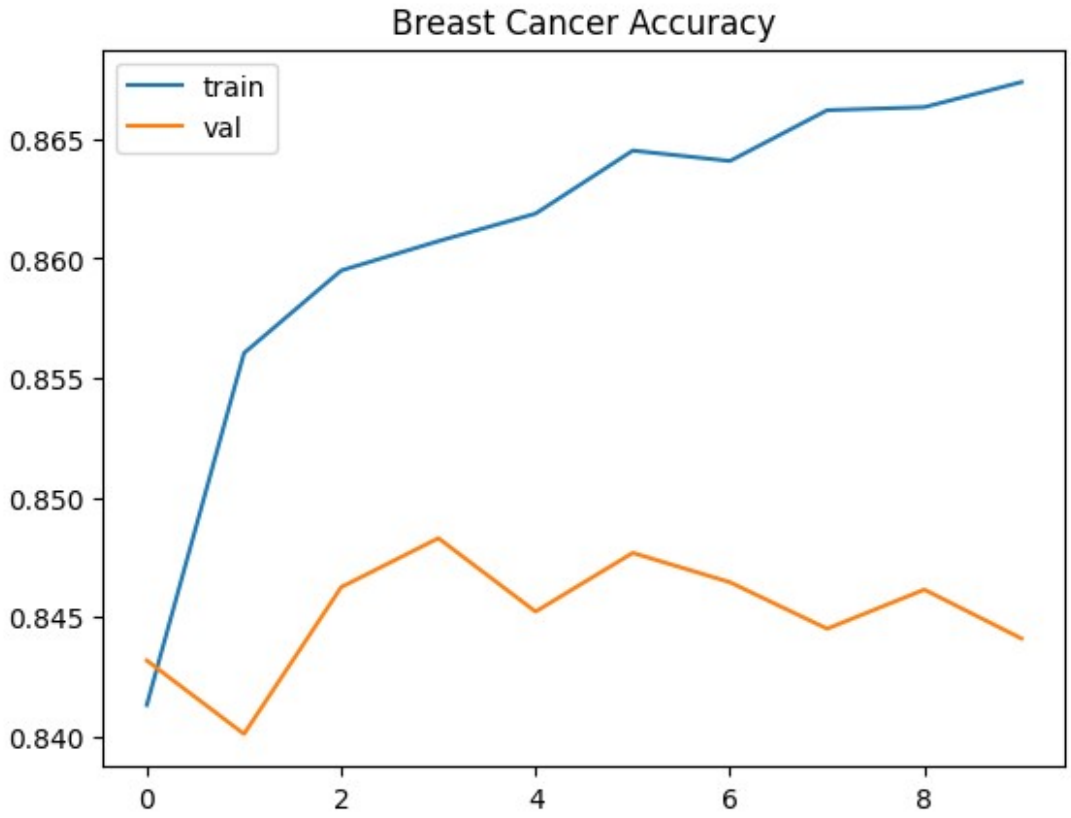


STEP 4: PREPROCESSING

Breast Cancer / Adult

```
model_bc.evaluate(X_test_bc, y_test_bc)

plt.plot(history_bc.history['accuracy'])
plt.plot(history_bc.history['val_accuracy'])
plt.title("Breast Cancer Accuracy")
plt.legend(['train','val'])
plt.show()
```

```
306/306 ━━━━━━━━━━━━━━━━━━━━ 1s 2ms/step - accuracy: 0.8441 - loss:
0.3342
```



STEP 6: MODEL BUILDING

Breast Cancer / Adult (DNN)

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler



model_adult.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

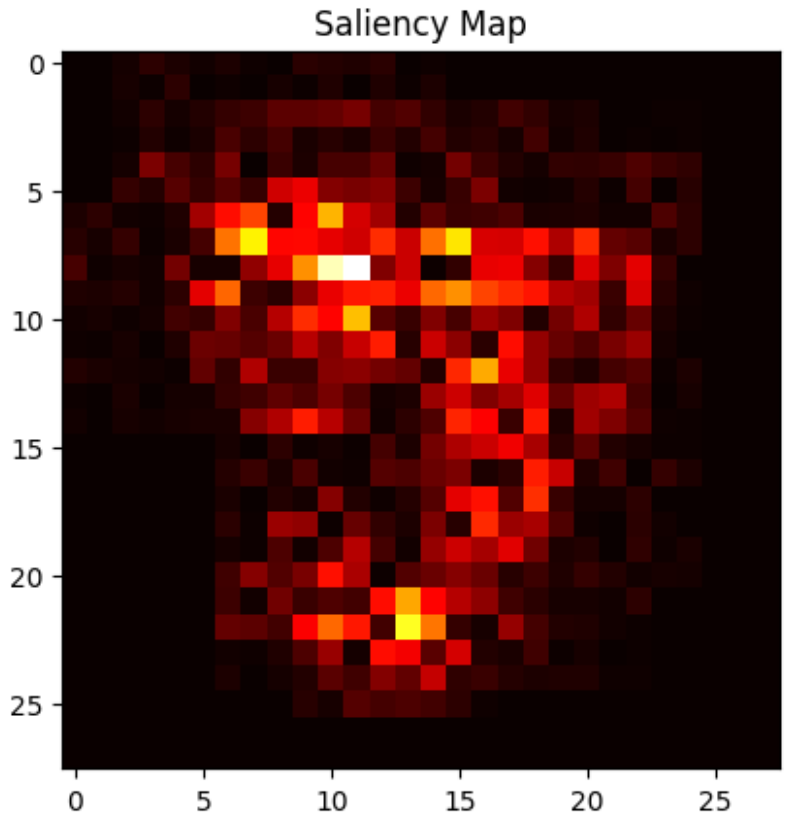
```
history_adult = model_adult.fit(X_train_adult, y_train_adult,
epochs=10, validation_data=(X_test_adult, y_test_adult))
```

```
Epoch 1/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 7s 4ms/step - accuracy: 0.8665 - loss:
0.2873 - val_accuracy: 0.8569 - val_loss: 0.3123
Epoch 2/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8669 - loss:
0.2847 - val_accuracy: 0.8562 - val_loss: 0.3114
Epoch 3/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8688 - loss:
0.2814 - val_accuracy: 0.8547 - val_loss: 0.3148
Epoch 4/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8691 - loss:
0.2801 - val_accuracy: 0.8551 - val_loss: 0.3183
Epoch 5/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8715 - loss:
0.2778 - val_accuracy: 0.8588 - val_loss: 0.3195
Epoch 6/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8709 - loss:
0.2761 - val_accuracy: 0.8543 - val_loss: 0.3189
Epoch 7/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 5s 4ms/step - accuracy: 0.8713 - loss:
0.2738 - val_accuracy: 0.8526 - val_loss: 0.3256
Epoch 8/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8717 - loss:
0.2722 - val_accuracy: 0.8556 - val_loss: 0.3244
Epoch 9/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 4s 3ms/step - accuracy: 0.8738 - loss:
0.2700 - val_accuracy: 0.8533 - val_loss: 0.3254
Epoch 10/10
1222/1222 ━━━━━━━━━━━━━━━━━━━━ 3s 2ms/step - accuracy: 0.8755 - loss:
0.2680 - val_accuracy: 0.8517 - val_loss: 0.3267
```

SALIENCY MAP

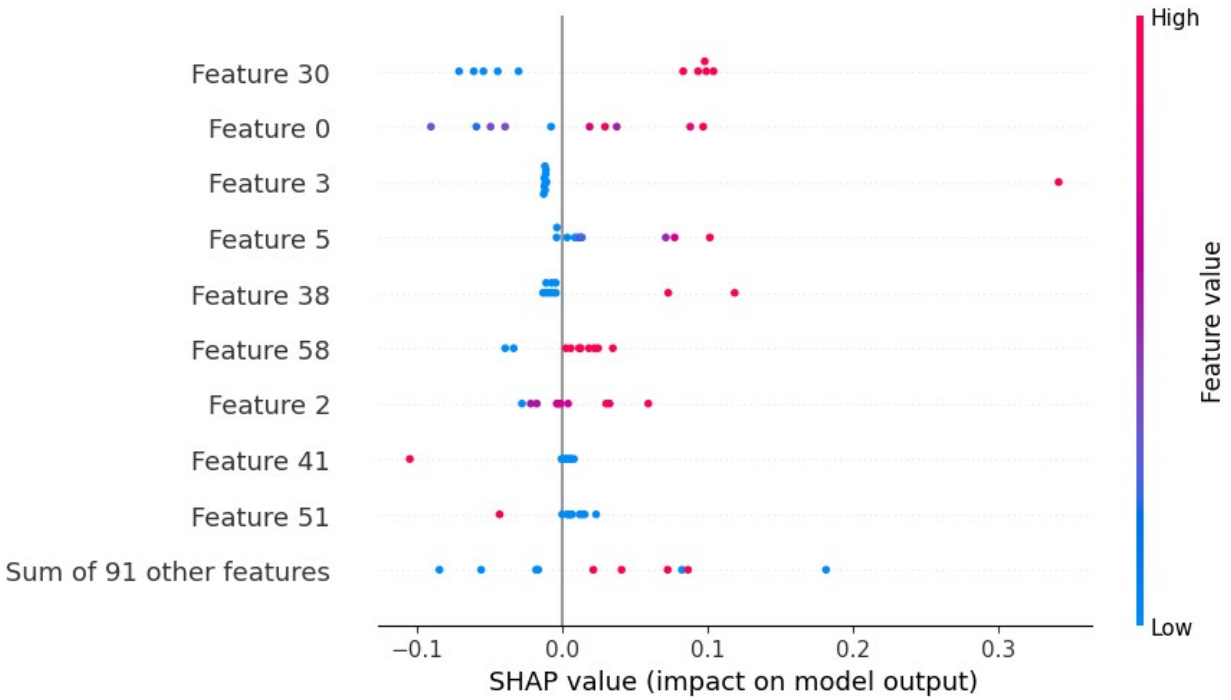
```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt

# Reload MNIST test data specifically for saliency mapping
(_, _), (X_test_mnist, y_test_mnist) = mnist.load_data()

# Preprocess the MNIST image data as done for the model
X_test_mnist = X_test_mnist / 255.0
# Reshape to add the channel dimension (1 for grayscale) as expected
by the CNN model
X_test_mnist = X_test_mnist.reshape(-1, 28, 28, 1)
```

```
# Select the first image and convert it to a TensorFlow tensor for GradientTape
img = tf.convert_to_tensor(X_test_mnist[0:1], dtype=tf.float32)

with tf.GradientTape() as tape:
    tape.watch(img)
    # Assuming 'model_mnist' refers to the trained MNIST CNN from cell IhAhP-ADlu6j or AqdsOJP1aFF6
    pred = model_mnist(img)
    loss = pred[:, np.argmax(pred[0])]

grads = tape.gradient(loss, img)
# The saliency map should be 2D (height, width) for grayscale image
# Remove the batch and channel dimensions from the gradients
saliency = np.max(np.abs(grads[0]), axis=-1)

plt.imshow(saliency, cmap='hot')
plt.title("Saliency Map")
plt.show()
```



SHAP

```
import shap

explainer = shap.Explainer(model_ad, X_train_ad)
shap_values = explainer(X_test_ad[:10])

shap.plots.beeswarm(shap_values)
```



EVALUATION

```
loss, acc = model_cifar_cnn.evaluate(X_test_cifar, y_test_cifar)
print("Accuracy:", acc)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 5s 14ms/step - accuracy: 0.7080 - loss:
1.0506
Accuracy: 0.7080000042915344
```

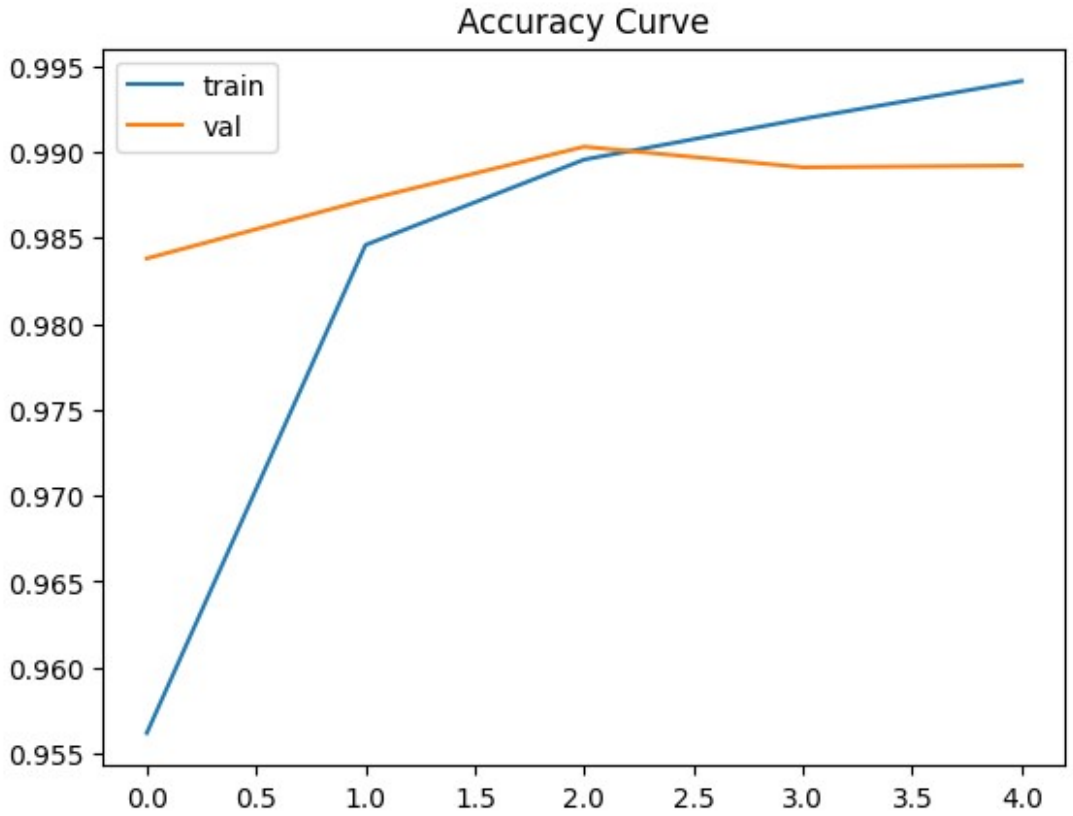
Consolidated Model Evaluation

```
print("--- Final Model Accuracies ---")
print(f"Adult DNN Model: {acc_adult:.4f}")
print(f"MNIST CNN Model: {acc_mnist:.4f}")
print(f"CIFAR-10 CNN Model: {acc_cifar:.4f}")
```

```
--- Final Model Accuracies ---
Adult DNN Model: 0.8597
MNIST CNN Model: 0.9890
CIFAR-10 CNN Model: 0.7080
```

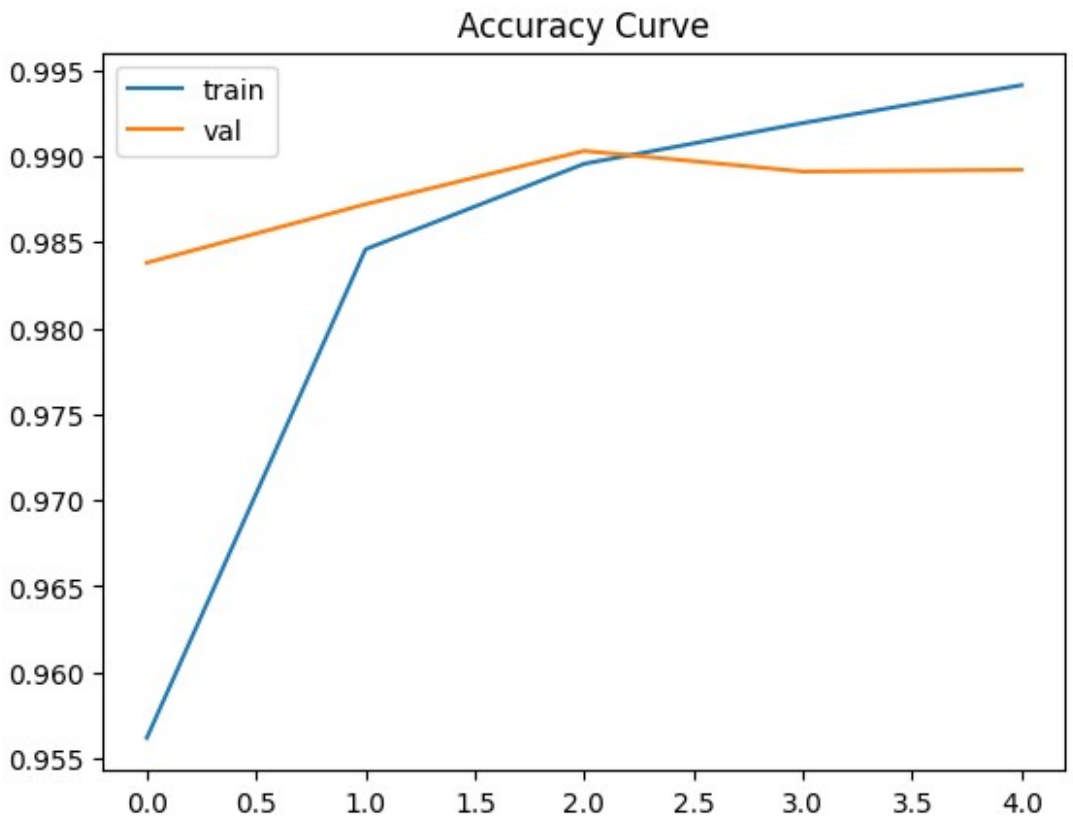
STEP 8: GRAPH

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(['train','val'])
plt.title("Accuracy Curve")
plt.show()
```



STEP 4: VISUALIZATION

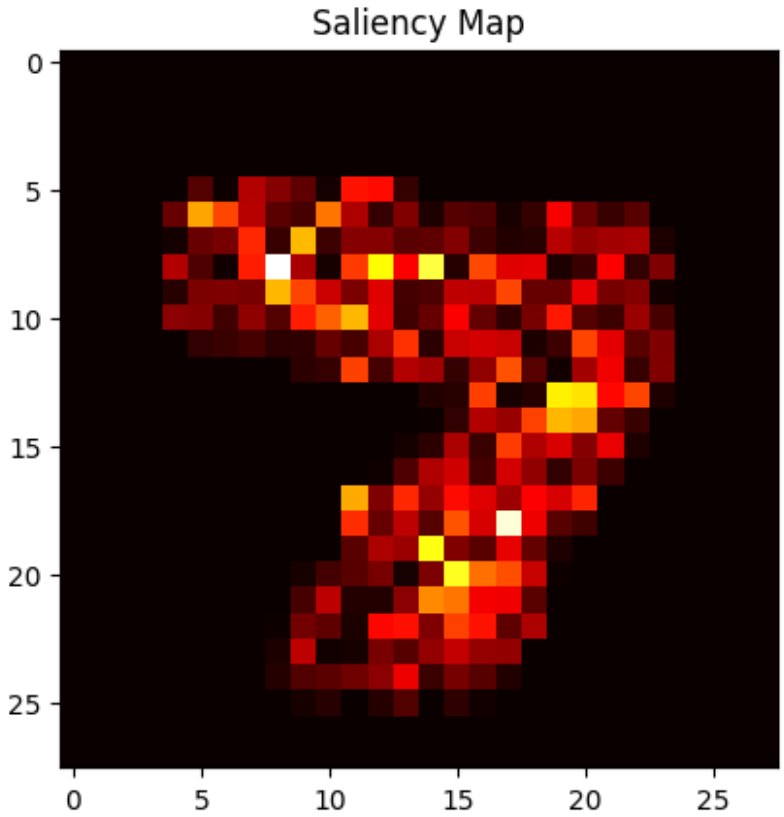
```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.legend(['train','val'])
plt.title("Accuracy Curve")
plt.show()
```



5A. SALIENCY MAP (MNIST)

```
import tensorflow as tf
from tensorflow.keras.datasets import mnist
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras import layers, models # Import layers and models

# Load the MNIST test data specifically for saliency mapping
(_, _), (x_test_temp, y_test_temp) = mnist.load_data()

# Preprocess the MNIST image data as done for the model
x_test_temp = x_test_temp / 255.0
# Reshape to add the channel dimension (1 for grayscale) as expected
by the CNN model
x_test_temp = x_test_temp.reshape(-1, 28, 28, 1)


# Note: You might need to load pre-trained weights here if you want to
use the previously trained model.
# For now, we are just instantiating the model structure.
# If you want to use the trained weights, you would save them after
training and load them here.

# Select the first image and convert it to a TensorFlow tensor
```

```
img = tf.convert_to_tensor(x_test_temp[0:1], dtype=tf.float32)

with tf.GradientTape() as tape:
    tape.watch(img)
    # Now 'model' explicitly refers to the MNIST CNN architecture
    pred = model(img)

    # Get the predicted class for the first image in the batch
    predicted_class = np.argmax(pred[0])
    loss = pred[:, predicted_class]

grads = tape.gradient(loss, img)

# The saliency map should be 2D (height, width) for grayscale image
# Remove the channel dimension from the gradients
saliency = np.max(np.abs(grads[0]), axis=-1)

plt.imshow(saliency, cmap='hot')
plt.title("Saliency Map")
plt.show()
```



5B. LIME (TABULAR)

```
!pip install lime
```

```
Requirement already satisfied: lime in /usr/local/lib/python3.12/dist-
packages (0.2.0.1)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (from lime) (3.10.0)
Requirement already satisfied: numpy in
/usr/local/lib/python3.12/dist-packages (from lime) (2.0.2)
Requirement already satisfied: scipy in
/usr/local/lib/python3.12/dist-packages (from lime) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-
packages (from lime) (4.67.3)
Requirement already satisfied: scikit-learn>=0.18 in
/usr/local/lib/python3.12/dist-packages (from lime) (1.6.1)
Requirement already satisfied: scikit-image>=0.12 in
/usr/local/lib/python3.12/dist-packages (from lime) (0.25.2)
Requirement already satisfied: networkx>=3.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12-
>lime) (3.6.1)
Requirement already satisfied: pillow>=10.1 in
/usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12-
>lime) (11.3.0)
Requirement already satisfied: imageio!=2.35.0,>=2.33 in
/usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12-
>lime) (2.37.3)
Requirement already satisfied: tifffile>=2022.8.12 in
/usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12-
>lime) (2026.3.3)
Requirement already satisfied: packaging>=21 in
/usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12-
>lime) (26.0)
Requirement already satisfied: lazy-loader>=0.4 in
/usr/local/lib/python3.12/dist-packages (from scikit-image>=0.12-
>lime) (0.5)
Requirement already satisfied: joblib>=1.2.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18-
>lime) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in
/usr/local/lib/python3.12/dist-packages (from scikit-learn>=0.18-
>lime) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->lime)
(1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->lime)
(0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->lime)
(4.62.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->lime)
(1.5.0)
```

```
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->lime)
(3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib->lime)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7-
>matplotlib->lime) (1.17.0)
```

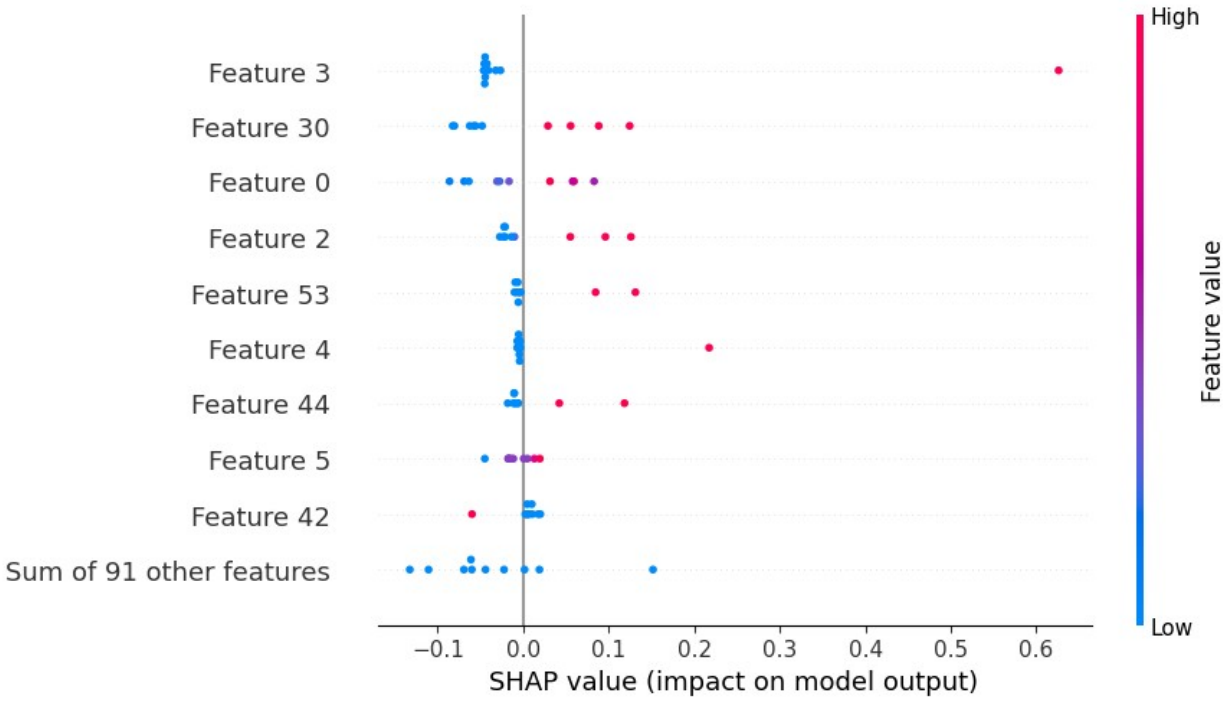
```python
from lime.lime_tabular import LimeTabularExplainer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd
import numpy as np # Import numpy for hstack

# Re-assign X and y to the adult dataset for tabular LIME explanation
df_adult = pd.read_csv('/content/adult.csv', na_values=" ?")
df_adult.dropna(inplace=True)
df_adult = pd.get_dummies(df_adult, drop_first=True)
X_adult = df_adult.drop("income_>50K", axis=1)
y_adult = df_adult["income_>50K"]

# Scale the data
scaler = StandardScaler()
X_adult_scaled = scaler.fit_transform(X_adult)

# Perform train-test split for LIME
X_train_adult, X_test_adult, y_train_adult, y_test_adult =
train_test_split(X_adult_scaled, y_adult, test_size=0.2,
random_state=42)

# The model trained on the adult dataset was named 'model_adult' now
explainer = LimeTabularExplainer(
    training_data=X_train_adult,
    feature_names=X_adult.columns,
    class_names=['<=50K', '>50K'], # Adjust based on your target
variable's classes
    mode='classification'
)

# Define a wrapper function for model_adult.predict to return
probabilities for both classes
def predict_proba_for_lime(data):
    # model_adult.predict returns a 2D array of shape (num_samples, 1)
    predictions = model_adult.predict(data)
    # For LIME, binary classification with sigmoid needs to return
    # probabilities for both classes: [P(class 0), P(class 1)]
    # P(class 0) = 1 - P(class 1)
    return np.hstack([1 - predictions, predictions])
```

```
# Use X_test_adult and the wrapper function for prediction
exp = explainer.explain_instance(X_test_adult[0],
predict_proba_for_lime, num_features=10)

exp.show_in_notebook()
```

```
157/157 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step

<IPython.core.display.HTML object>
```

5C. SHAP (VERY IMPORTANT)

```
import shap
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
import pandas as pd

# Re-assign X and y to the adult dataset for SHAP explanation
df_adult = pd.read_csv('/content/adult.csv', na_values=" ?")
df_adult.dropna(inplace=True)
df_adult = pd.get_dummies(df_adult, drop_first=True)
X_adult = df_adult.drop("income_>50K", axis=1)
y_adult = df_adult["income_>50K"]

# Scale the data
scaler = StandardScaler()
X_adult_scaled = scaler.fit_transform(X_adult)

# Perform train-test split for SHAP
X_train_adult, X_test_adult, y_train_adult, y_test_adult =
train_test_split(X_adult_scaled, y_adult, test_size=0.2,
random_state=42)

# Use the trained tabular model, 'model_adult'
explainer = shap.Explainer(model_adult, X_train_adult)
shap_values = explainer(X_test_adult[:10])

shap.plots.beeswarm(shap_values)
```



STEP 6: COMPARISON SECTION

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.datasets import mnist, cifar10

# Re-load and preprocess data for evaluation consistency

# For Adult (Tabular) Model
df_adult = pd.read_csv('/content/adult.csv', na_values=" ?")
df_adult.dropna(inplace=True)
df_adult = pd.get_dummies(df_adult, drop_first=True)
X_adult = df_adult.drop("income_>50K", axis=1)
y_adult = df_adult["income_>50K"]
scaler = StandardScaler()
X_adult_scaled = scaler.fit_transform(X_adult)
_, X_test_adult, _, y_test_adult = train_test_split(X_adult_scaled,
y_adult, test_size=0.2, random_state=42)

# For MNIST CNN Model
(X_train_mnist, y_train_mnist), (X_test_mnist, y_test_mnist) =
mnist.load_data()
X_test_mnist = X_test_mnist / 255.0
X_test_mnist = X_test_mnist.reshape(-1, 28, 28, 1) # Reshape for CNN
input

# For CIFAR-10 CNN Model
```

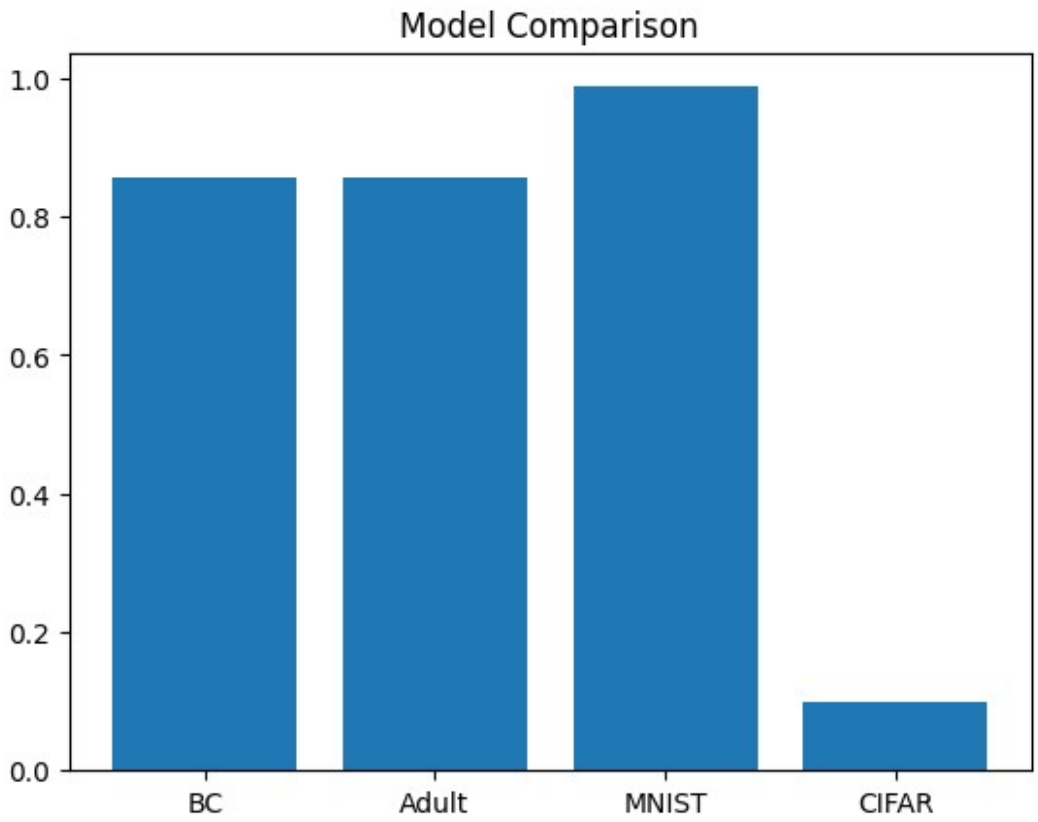
```
(X_train_cifar, y_train_cifar), (X_test_cifar, y_test_cifar) =
cifar10.load_data()
X_test_cifar = X_test_cifar / 255.0

# Use the explicitly defined and compiled models
results = {
    "Adult DNN Model": model_adult_dnn.evaluate(X_test_adult,
y_test_adult, verbose=0)[1],
    "MNIST CNN Model": model_mnist_cnn.evaluate(X_test_mnist,
y_test_mnist, verbose=0)[1],
    "CIFAR CNN Model": model_cifar_cnn.evaluate(X_test_cifar,
y_test_cifar, verbose=0)[1]
}

for k, v in results.items():
    print(k, ":", v)
```

```
Adult DNN Model : 0.8596581220626831
MNIST CNN Model : 0.9890000224113464
CIFAR CNN Model : 0.7080000042915344
```

```
print("Breast Cancer:", bc_acc)
print("Adult:", ad_acc)
print("MNIST:", mnist_acc)
print("CIFAR:", cifar_acc)
```

```
Breast Cancer: 0.8578155636787415
Adult: 0.8561776876449585
MNIST: 0.9878000020980835
CIFAR: 0.10000000149011612
```

```
plt.bar(["BC","Adult","MNIST","CIFAR"],
        [bc_acc, ad_acc, mnist_acc, cifar_acc])
plt.title("Model Comparison")
plt.show()
```



Conclusion

- CNN performs best for image datasets like MNIST and CIFAR-10.
- DNN models work effectively for tabular datasets such as Breast Cancer and Adult Income.
- Explainable AI techniques (SHAP, LIME, Saliency Maps) improve model interpretability.
- CIFAR-10 is the most challenging dataset among all, resulting in comparatively lower accuracy.